



Streaming Job

Ingestion to raw table



1<sup>st</sup> Oct

14<sup>th</sup> Oct

17<sup>th</sup> Oct

1. Deploy new Job
2. Start the Job

1. Stop the Job
2. Deploy New Feature
3. Restart the Job from the point it was stopped

1. Stop the Job
2. Deploy Fix
3. Restart the Job from 14<sup>th</sup> October

Checkpoint allows you to restart from the point of stop

startingTimestamp + Empty Checkpoint allows you to restart from any date/time

How to implement idempotence?



## 12-idempotent-kafka-to-bronze Python

File Edit View Run Help Last edit was 11 minutes ago Provide feedback

Run all

Connect

Share

Publish

```
        "topic", "timestamp")
    )

def upsert(self, invoices_df, batch_id):
    invoices_df.createOrReplaceTempView("invoices_df_temp_view")
    merge_statement = """MERGE INTO invoices_bz s
        USING invoices_df_temp_view t
        ON s.value == t.value AND s.timestamp == t.timestamp
        WHEN MATCHED THEN
            UPDATE SET *
        WHEN NOT MATCHED THEN
            INSERT *
    """

def process(self, startingTime = 1):
    print(f"Starting Bronze Stream...", end='')
    kafka_df = self.ingestFromKafka(startingTime)
    invoices_df = self.getInvoices(kafka_df)
    sQuery = ( invoices_df.writeStream
                .queryName("bronze-ingestion")
                .foreachBatch(self.upsert)
                .option("checkpointLocation", f"{self.base_data_dir}/checkpoint/invoices_bz")
                .outputMode("append")
            )
    print("Done")
    return sQuery
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text



## 12-idempotent-kafka-to-bronze Python ▾

File Edit View Run Help Last edit was 12 minutes ago Provide feedback

▶ Run all

● Connect ▾

Share

Publish

```
USING invoices_df_temp_view t
ON s.value == t.value AND s.timestamp == t.timestamp
WHEN MATCHED THEN
UPDATE SET *
WHEN NOT MATCHED THEN
INSERT *
```

```
def process(self, startingTime = 1):
    print(f"Starting Bronze Stream...", end='')
    kafka_df = self.ingestFromKafka(startingTime)
    invoices_df = self.getInvoices(kafka_df)
    sQuery = invoices_df.writeStream
        .queryName("bronze-ingestion")
        .foreachBatch(self.upsert)
        .option("checkpointLocation", f"{self.base_data_dir}/checkpoint/invoices_bz")
        .outputMode("append")
        .start()

    print("Done")
    return sQuery
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text



### 13-idempotent-kafka-to-bronze-test-suite Python ▾

File Edit View Run Help Last edit was 5 minutes ago Provide feedback

▶ Run all

● Connect ▾

Share

Publish

```
print("Done.")
```

```
def runTests(self):
```

```
    self.cleanTests()
```

```
    bzStream = Bronze()
```

```
    value_schema = bzStream.getSchema()
```

```
    spark.sql(f"CREATE TABLE invoices_bz (key STRING, value STRUCT<{value_schema}>, topic STRING, timestamp TIMESTAMP)")
```

```
    print("Testing Scenario - Start from beginneing on a new checkpoint...")
```

```
    bzQuery = bzStream.process()
```

```
    self.waitForMicroBatch()
```

```
    bzQuery.stop()
```

```
    self.assertResult(30)
```

```
    print("Validation passed.\n")
```

```
    print("Testing Scenarion - Restart from where it stopped on the same checkpoint...")
```

```
    bzQuery = bzStream.process()
```

```
    self.waitForMicroBatch()
```

```
    bzQuery.stop()
```

```
    self.assertResult(30)
```

```
    print("Validation passed.\n")
```

```
    print("Testing Scenario - Start from 1697945539000 on a new checkpoint...")
```

```
    dbutils.fs.rm(f"{self.base_data_dir}/chekpoint/invoices_bz", True)
```

```
    bzQuery = bzStream.process(1697945539000)
```

```
    self.waitForMicroBatch()
```

```
    bzQuery.stop()
```

```
    self.assertResult(40)
```

```
    print("Validation passed.\n")
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text



## 12-idempotent-kafka-to-bronze Python

File Edit View Run Help Last edit was 13 minutes ago Provide feedback

Run all

my cluster

Share

Publish

```
def upsert(self, invoices_df, batch_id):
    invoices_df.createOrReplaceTempView("invoices_df_temp_view")
    merge_statement = """MERGE INTO invoices_bz s
        USING invoices_df_temp_view t
        ON s.value == t.value AND s.timestamp == t.timestamp
        WHEN MATCHED THEN
            UPDATE SET *
        WHEN NOT MATCHED THEN
            INSERT *
    """
    invoices_df._jdf.sparkSession().sql(merge_statement)

def process(self, startingTime = 1):
    print(f"Starting Bronze Stream...", end='')
    kafka_df = self.ingestFromKafka(startingTime)
    invoices_df = self.getInvoices(kafka_df)
    sQuery = ( invoices_df.writeStream
        .queryName("bronze-ingestion")
        .foreachBatch(self.upsert)
        .option("checkpointlocation", f"{self.base_data_dir}/checkpoint/invoices_bz")
        .outputMode("append")
        .start()
    )
    print("Done")
    return sQuery
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

